Final Project

Estimated Duration: 45 minutes

Learning Objectives

After completing this project, you will be able to:

- Create classes and objects with OOP concepts such as encapsulation and polymorphism
- Use collections to store objects in memory
- Create, store, retrieve and manipulate Date and Time objects
- Handle File I/O

About the course project

Welcome to the final project for `Object Oriented Programming in Java`. In this project, you will apply the knowledge and skills learned in this course to a simulated scenario.

In this task, you will create a console application for a mood tracker app, which will allow you to create a Mood object with attributes, name, date, time, and notes. The tasks in this hands-on project correspond to the activities performed by a Java Developer who is creating a stand-alone console application.

You are currently viewing this lab in a Cloud based Integrated Development Environment (Cloud IDE). It is a fully-online integrated development environment that is pre-installed with JDK 21, allowing you to code, develop, and learn in one location.

Mood class

1. Create a project directory by running the following command.

   ```
   mkdir final_proj
   ```

2. Run the following code to create the directory structure.

```
mkdir -p final_proj/src
mkdir -p final_proj/classes
mkdir -p final_proj/test
cd final_proj
```

3. Now, create a file named `Mood.java` inside the src directory.

```
touch /home/project/final_proj/src/Mood.java
```

4. Select the following button to open the file for editing.

Open **Mood.java** in IDE

Start adding the code

1. Create a public class with the attributes, constructor, setters, getters, toString, and equals method. The `date` attribute should be set to default current date. The `time` attribute should be set to default `LocalTime.MIDNIGHT`. Overload the constructor to take the following parameters:

- Mood(String name)
- Mood(String name, LocalDate date)
- Mood(String name, LocalDate date, LocalTime time)
- Mood(String name, String notes)
- Mood(String name, LocalDate date, String notes)
- Mood(String name, LocalDate date, LocalTime time, String notes)

▼ Click here for sample

```
import java.time.LocalDate;
import java.time.LocalTime;
public class Mood {
    private String name;
    private LocalDate date = LocalDate.now();
    private LocalTime time = LocalTime.MIDNIGHT;
    private String notes;
    public Mood(String name) {
        this.name = name;
    }
    public Mood(String name, LocalDate date) {
        this.name = name;
        this.date = date;
    }
    public Mood(String name, LocalDate date, LocalTime time) {
        this.name = name;
        this.date = date;
        this.time = time;
    }
    public Mood(String name, String notes) {
        this.name = name;
        this.notes = notes;
    }
    public Mood(String name, LocalDate date, String notes) {
        this.name = name;
        this.date = date;
        this.notes = notes;
    }
    public Mood(String name, LocalDate date, LocalTime time, String notes) {
        this.name = name;
        this.date = date;
```

```
            this.time = time;
            this.notes = notes;
        }
        public String getName() {
            return this.name;
        }
        public LocalDate getDate() {
            return this.date;
        }
        public void setDate(LocalDate date) {
            this.date = date;
        }
        public LocalTime getTime() {
            return this.time;
        }
        public void setTime(LocalTime time) {
            this.time = time;
        }
        public String getNotes() {
            return this.notes;
        }
        public void setNotes(String notes) {
            this.notes = notes;
        }
        public String toString() {
            return name + " - " + date + " " + time + "\n" +notes;
        }
        public boolean equals(Mood mood) {
            if(mood.name.equalsIgnoreCase(this.name) &&
                mood.getDate().equals(this.date) &&
                mood.getTime().equals(this.time)) {
                    return true;
                } else {
                    return false;
                }
        }
    }
```

2. Compile `Mood.java` and verify the class is created within the `classes` directory.

3. Set the classpath. **Please ensure that this step is complete. Else your classes will not compile as expected**.

Create InvalidMoodException

1. Now create a file named `InvalidMoodException.java` inside the `src` directory.

```
touch /home/project/final_proj/src/InvalidMoodException.java
```

2. Select the following button to open the file for editing.

Open **InvalidMoodException.java** in IDE

3. Add appropriate code in InvalidMoodException.java.

▼ Click here for sample

```
public class InvalidMoodException extends Exception {
    public InvalidMoodException() {
        super();
    }
}
```

4. Compile `InvalidMoodException.java` and verify the class is created within the `classes` directory.

Create Mood Tracker app

1. Now create a file named `MoodTracker.java` inside the `src` directory.

```
touch /home/project/final_proj/src/MoodTracker.java
```

2. Select the following button to open the file for editing.

Open **MoodTracker.java** in IDE

3. Create a public class with a public main method.

   Note: Remember to add all the import statements as and when you are using any classes that are external.

▼ Click here for the sample code.

```
public class MoodTracker {
    public static void main(String s[]) {
    }
}
```

4. Create an ArrayList that can store `Mood` objects.
5. Import scanner from the `util` package and create an object of it to read from the console in the MoodTracker application.

Infinite loop with Menu Options

1. Select the following button to open the file for editing if it is not open already.

Open **MoodTracker.java** in IDE

2. Create an infinite loop that runs as long as the user wants it to. The code within the loop should present the user with options. The loop should exit when the user inputs `Exit`.

▼ Click here for sample code.

```
while(true) {
        System.out.println("Press 'a' to add mood\n" +
                           "'d' to delete mood(s)\n" +
                           "'e' to edit mood\n" +
                           "'s' to search for moods\n" +
                           "'M' to get all moods\n" +
                           "'w' to write the moods to a file\n" +
                           "Type 'Exit' to exit");
        String menuOption = scanner.nextLine();
        switch(menuOption) {
            case "a":       //add code to add mood
                        continue;
            case "d":       //add code to delete mood
                        continue;
            case "e":       //add code to edit mood
                        continue;
            case "s":       //add code to search mood
                        continue;
            case "M":       //add code to get all moods
                        continue;
            case "w":       //add code to write mood to a file
                        continue;
            case "Exit":    System.out.println("Thank you for using the MoodTracker. Goodbye!");
                            break;
            default:        System.out.println("Not a valid input!");
```

```
                                continue;
                        }
        }
```

Add a mood

1. Get input for mood name from the user. Get date and time. You can allow user to press enter to skip entering the date and time.
2. Check mood validity. The mood on a particular date, at a given time cannot change. If a mood on the same date and time exists, throw an exception. If the mood is validated, store it.

▼ Click here for code sample.

To check if mood is valid:

```
public static boolean isMoodValid(Mood mood, ArrayList<Mood> moodsList) throws InvalidMoodException {
    for(Mood tempMood: moodsList) {
        if (tempMood.equals(mood)) {
            throw new InvalidMoodException();
        }
    }
    return true;
}
```

To add the mood to the list, after checking validity:

```
System.out.println("Enter the mood name");
String moodName = scanner.nextLine();
System.out.println("Are you tracking the mood for a current day? y/n");
String isForCurrentDate = scanner.nextLine();
Mood moodToAdd = null;
if(isForCurrentDate.equalsIgnoreCase("n")) {
    try {
        System.out.println("Input the date in MM/dd/yyyy format:");
        String moodDateStr = scanner.nextLine();
        DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("MM/dd/yyyy");
        LocalDate moodDate = LocalDate.parse(moodDateStr, dateFormatter);
        System.out.println("Input the time in HH:mm:ss format:");
        String moodTimeStr = scanner.nextLine();
        DateTimeFormatter timeFormatter = DateTimeFormatter.ofPattern("HH:mm:ss");
        LocalTime moodTime = LocalTime.parse(moodTimeStr, timeFormatter);
        System.out.println("Add notes about this mood");
        String moodNotes = scanner.nextLine();
        if(moodNotes.strip().equalsIgnoreCase("")) {
            moodToAdd = new Mood(moodName, moodDate, moodTime);
        } else {
            moodToAdd = new Mood(moodName, moodDate, moodTime, moodNotes);
        }
    } catch (DateTimeParseException dfe) {
        System.out.println("Incorrect format of date or time. Cannot create mood.\n"+dfe);
        continue;
    }
} else {
    System.out.println("Add notes about this mood");
    String moodNotes = scanner.nextLine();
    if(moodNotes.strip().equalsIgnoreCase("")) {
        moodToAdd = new Mood(moodName);
    } else {
        moodToAdd = new Mood(moodName, moodNotes);
    }
}
try {
    boolean isValid = isMoodValid(moodToAdd, moodsList);
    if(isValid) {
        moodsList.add(moodToAdd);
        System.out.println("The mood has been added to the tracker");
        continue;
    }
} catch(InvalidMoodException ime) {
    System.out.println("The mood is not valid");
}
```

Delete moods by date or delete a specific mood

1. For the delete option, provide the user with two further options.

- The first option is to delete the moods by date. Get the date as an input from user and delete all moods tracked for that day.
- The second option is to delete one mood, based on the name, date and time. Get the name, date, and time as an input from user and delete the mood.

▼ Click here for sample code.

To delete by date:

```
public static boolean deleteMoods(LocalDate moodDate, ArrayList<Mood> moodsList) {
    boolean removed = false;
    for(Mood tempMood: moodsList) {
        if (tempMood.getDate().equals(moodDate)) {
            moodsList.remove(tempMood);
            removed = true;
        }
    }
    return removed;
}
```

To delete the mood:

```
public static boolean deleteMood(Mood mood, ArrayList<Mood> moodsList) {
    for(Mood tempMood: moodsList) {
        if (tempMood.equals(mood)) {
            moodsList.remove(tempMood);
            return true;
        }
    }
    return false;
}
```

To delete the mood from the list:

```
System.out.println("Enter '1' to delete all moods by date\n"+
                   "Enter '2' to delete a specific mood");
String deleteVariant = scanner.nextLine();
if(deleteVariant.equals("1")) {
    try {
        System.out.println("Input the date in MM/dd/yyyy format:");
        String moodDateStr = scanner.nextLine();
        DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("MM/dd/yyyy");
        LocalDate moodDate = LocalDate.parse(moodDateStr, dateFormatter);
        boolean areMoodsDeleted = deleteMoods(moodDate, moodsList);
        if(areMoodsDeleted) {
            System.out.println("The moods have been deleted");
        } else {
            System.out.println("No matching moods found");
        }
    } catch (DateTimeParseException dfe) {
        System.out.println("Incorrect format of date. Cannot delete mood.");
        continue;
    }
} else if (deleteVariant.equals("2")) {
    try {
        System.out.println("Enter the mood name");
```

```
            moodName = scanner.nextLine();
            System.out.println("Input the date in MM/dd/yyyy format:");
            String moodDateStr = scanner.nextLine();
            DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("MM/dd/yyyy");
            LocalDate moodDate = LocalDate.parse(moodDateStr, dateFormatter);
            System.out.println("Input the time in HH:mm:ss format:");
            String moodTimeStr = scanner.nextLine();
            DateTimeFormatter timeFormatter = DateTimeFormatter.ofPattern("HH:mm:ss");
            LocalTime moodTime = LocalTime.parse(moodTimeStr, timeFormatter);
            Mood delMood = new Mood(moodName, moodDate, moodTime);
            boolean isMoodDeleted = deleteMood(delMood, moodsList);
            if(isMoodDeleted) {
                System.out.println("The mood has been deleted");
            } else {
                System.out.println("No matching mood found");
            }
        } catch (DateTimeParseException dfe) {
            System.out.println("Incorrect format of date or time. Cannot delete mood.");
            continue;
        }
    }
```

Edit a specific mood

1. Get the mood, date and time as a user input and allow user to edit the notes.

▼ Click here for sample code.

Edit the mood:

```
    public static boolean editMood(Mood moodToEdit, ArrayList<Mood> moodsList) {
        for(Mood tempMood: moodsList) {
            if (tempMood.equals(moodToEdit)) {
                tempMood.setNotes(moodToEdit.getNotes());
```

```
                return true;
            }
        }
        return false;
    }
```

When the user selects the edit option:

```
    Mood moodToEdit = null;
    try {
        System.out.println("Enter the mood name");
        moodName = scanner.nextLine();
        System.out.println("Input the date in MM/dd/yyyy format:");
        String moodDateStr = scanner.nextLine();
        DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("MM/dd/yyyy");
        LocalDate moodDate = LocalDate.parse(moodDateStr, dateFormatter);
        System.out.println("Input the time in HH:mm:ss format:");
        String moodTimeStr = scanner.nextLine();
        DateTimeFormatter timeFormatter = DateTimeFormatter.ofPattern("HH:mm:ss");
        LocalTime moodTime = LocalTime.parse(moodTimeStr, timeFormatter);
        System.out.println("Add new notes about this mood");
        String moodNotes = scanner.nextLine();
        if(moodNotes.strip().equalsIgnoreCase("")) {
            System.out.println("No notes entered");
            continue;
        } else {
            moodToEdit = new Mood(moodName, moodDate, moodTime, moodNotes);
            boolean isMoodEdited = editMood(moodToEdit, moodsList);
            if(isMoodEdited) {
                System.out.println("The mood has been successfully edited");
            } else {
                System.out.println("No matching mood could be found");
            }
        }
    } catch (DateTimeParseException dfe) {
        System.out.println("Incorrect format of date or time. Cannot create mood.");
        continue;
```

```
}
```

Search moods by date or search a specific mood

1. For the search option, provide the user with two additional options.

- The first option is to get moods for a date. Get the date as an input from user and retrieve all moods tracked for that day.
- The second option is to get one mood, given the name, date and time. Get the name, date and time as an input from user and retrieve the mood that matches that.

▼ Click here for sample code

Search for moods by date

```
public static void searchMoods(LocalDate moodDate, ArrayList<Mood> moodsList) {
    boolean found = false;
    for(Mood tempMood: moodsList) {
        if (tempMood.getDate().equals(moodDate)) {
            found = true;
            System.out.println(tempMood);
        }
    }
    if(!found) {
        System.out.println("No matching records could be found!");
    }
}
```

Search for a specific mood

```
public static void searchMood(Mood mood, ArrayList<Mood> moodsList) {
    boolean found = false;
    for(Mood tempMood: moodsList) {
        if (tempMood.equals(mood)) {
            found = true;
            System.out.println(tempMood);
        }
    }
    if(!found) {
        System.out.println("No matching records could be found!");
    }
}
```

Hanlde search options

```
System.out.println("Enter '1' to search for all moods by date\n"+
                                                "Enter '2' to search for a specific mood");
String searchVariant = scanner.nextLine();
if(searchVariant.equals("1")) {
    try {
        System.out.println("Input the date in MM/dd/yyyy format:");
        String moodDateStr = scanner.nextLine();
        DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("MM/dd/yyyy");
        LocalDate moodDate = LocalDate.parse(moodDateStr, dateFormatter);
        searchMoods(moodDate, moodsList);
    } catch (DateTimeParseException dfe) {
        System.out.println("Incorrect format of date. Cannot search mood.");
        continue;
    }
```

```
    } else if (searchVariant.equals("2")) {
        try {
            System.out.println("Enter the mood name");
            moodName = scanner.nextLine();
            System.out.println("Input the date in MM/dd/yyyy format:");
            String moodDateStr = scanner.nextLine();
            DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("MM/dd/yyyy");
            LocalDate moodDate = LocalDate.parse(moodDateStr, dateFormatter);
            System.out.println("Input the time in HH:mm:ss format:");
            String moodTimeStr = scanner.nextLine();
            DateTimeFormatter timeFormatter = DateTimeFormatter.ofPattern("HH:mm:ss");
            LocalTime moodTime = LocalTime.parse(moodTimeStr, timeFormatter);
            Mood delMood = new Mood(moodName, moodDate, moodTime);
            searchMood(delMood, moodsList);
        } catch (DateTimeParseException dfe) {
            System.out.println("Incorrect format of date or time. Cannot search mood.");
            continue;
        }
    }
```

Get all the moods

1. Add code to get all moods that are stored in the tracker and print them on the console.

▼ Click here for sample code.

```
for(Mood moodObj: moodsList) {
    System.out.println(moodObj);
}
```

Write the moods to a file

1. Add code to write all the ArrayList moods to a file named `moodtracker.txt`.

▼ Click here for the sample code.

```
try (PrintWriter writer = new PrintWriter(new FileWriter("Moods.txt"))) {
    for (Mood mood : moodsList) {
        writer.println(mood+"\n\n");
    }
    System.out.println("The entries are written to a file");
} catch (IOException e) {
    System.err.println("Error writing to file: " + e.getMessage());
}
```

Make sure all the required packages are imported. Otherwise the compilation would fail.

▼ Click here for all the classes you need to import

```
import java.util.Scanner;
import java.util.ArrayList;
import java.io.PrintWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeParseException;
import java.time.format.DateTimeFormatter;
```

Compile and run code

1. Set the classpath to point to `final_proj/classes`.

   ```
   export CLASSPATH=$CLASSPATH:/home/project/final_proj/classes
   ```

2. Compile the file `Mood.java`.

   ```
   javac -d final_proj/classes final_proj/src/Mood.java
   ```

3. Compile the file `MoodTracker.java`.

```
javac -d final_proj/classes final_proj/src/MoodTracker.java
```

4. Run the java program.

```
java MoodTracker
```

Author(s)

Lavanya